Politecnico di Torino

Ingegneria Informatica (Computer Engineering) LM-32(DM270)

A.a. 2025/2026

Graduation Session March 2026

Managing large hordes of NPCs in Unreal Engine 5

An analysis of Mass ECS system

Supervisors:

Francesco Strada

Corrado Raffaelli

Candidate:

Martina Plumari

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Figures

Chapter 1

Introduction

1.1 Goal

The primary objective of this thesis is to design and implement a highly scalable NPC simulation using the Unreal Engine Mass Framework, targeting a performance threshold of thousands of concurrent entities on mid-to-high-range consumer hardware. To achieve this, the study focuses on developing a robust architectural pipeline that moves beyond passive background elements, ensuring that agents remain interactable and visually convincing through dynamic animation state management. A core technical challenge involves the implementation of a hybrid navigation system capable of reconciling global path-following with local, reactive steering for moving targets across non-planar terrain. Furthermore, the project serves as a critical feasibility study to evaluate the production readiness of Mass's ECS-based ecosystem. By identifying the current limitations in out-of-the-box avoidance, documentation accessibility, and the technical barrier for non-programming disciplines, this thesis aims to provide a definitive recommendation for the framework's adoption within a professional game development pipeline.

1.2 Target Platform Definition

The target used for performance considerations is a laptop with the following hardware specifications:

- CPU: 12th Generation Intel Core i7-12700H
- RAM: 16GB DDR5-4800 MHz
- GPU: NVIDIA GeForce RTX 3050 Ti (4 GB dedicated memory)

1.3 Thesis Structure

Chapter 2

Entity Component System

The Entity Component system (ECS) is a software architectural pattern specifically engineered to facilitate high-performance simulations. Unlike traditional game development frameworks that rely on deep object hierarchies, ECS is built upon the Data-Oriented Design (DOD) paradigm.

At its core, ECS shifts the focus from "what an object is" to "what data it possesses". By decoupling data from behavior, it enables the construction of complex simulations using granular, reusable modules rather than monolithic, rigid classes.

This chapter first analyses the Entity–Component–System architectural pattern and the reasons it can offer performance advantages over the traditional object-oriented paradigm. It then examins the trade-offs of this type of design compared to the object-oriented one.

2.1 Object-Oriented vs Data-Oriented Design

To understand why ECS has become the industry standard for mass-scaled simulations, we must compare it to the traditional Object-Oriented Design (OOD).

In OOD, functionality is typically extended through inheritance, making the resulting objects inextricably bound to their ancestors through a hard-coded relationship. As a project grows, this can lead to deep and rigid class hierarchies. For instance, when a Soldier class inherits from a Human class, it does not merely get access to human traits, but becomes physically coupled to the entire memory footprint and logic chain of the parent. On the other hand, DOD favors composition; in this case, rather than defining what an entity is (a Soldier), we define what it has (Transform, Health, Ammo). This approach allows us to avoid being forced to choose between two unrelated classes and enables architectural flexibility since the entity is simply a dynamic collection of data components assigned at runtime.

The primary driver for DOD is data locality. Modern CPU speeds vastly outpace memory access times; consequently, performance is often throttled by "memory latency", the delaay incurred when fetching data from RAM. In Object-Oriented Design, objects are frequently scattered throughout memory. Accessing them requires "pointer-chasing" through class hierarchies, leading to frequent cache misses and significant performance penalties in contemporary CPU architectures. On the other hand, in Data-Oriented Design components of an identical type are stored contiguously in memory. When a system processes these components the CPU can accurately predict the next data address and pre-load it into the cache. This maximizes cache coherency, facilitating rapid, linear iteration over thousands of entities.

2.2 The Three Pillars of ECS

As the name suggests, the ECS pattern consists of three main functional layers:

- Entity: it is a unique identifier serving as a handle that associates a set of components. It contains no intrinsic data or behavior. Its sole purpose is to provide a unique index to which stateful data can be mapped;
- Component: granular data container that encapsulate state but contain no logic. As "plain old data" structures, components are stored in dense arrays to optimize memory throughput;
- System: it represents the logic layer of the architecture. Systems are independent units that operate over filtered collections of components by means of declarative queries. Because systems operate on dense arrays with explicit data dependencies, they are inherently suited for multi-threaded execution, allowing the engine to distribute workloads across CPU cores with minimal synchronization overhead.

2.2.1 Trade-offs Analysis

While ECS provides the performance requisite for systems like UE Mass, it introduces specific architectural trade-offs:

- Architectural Complexity: initial planning requires a more rigorous design of component interfaces and system boundaries compared to the more intuitive Object based approach of OOD;
- Decoupling Overhead: the radical separation of data can lead to "component bloat" if the granularity is not managed carefully, potentially resulting in repetitive or trivial data structures.

- Mentality Shift: transitioning from an object-centric logic to Data-Oriented paradigm requires a fundamental shift in how developers conceptualize program flow and state management. This shift takes time, and the steep learning curve may not be strategically feasible for all production timelines or team structures.

Chapter 3

Unreal Engine Mass

While the Entity Component System is a theoretical pattern, Mass is Unreal Engine's specific framework for data-oriented calculations and high-performance simulations. It does not replace the traditional Actor-based system; instead, it provides a parallel infrastructure capable of processing a large number of entities with minimal CPU overhead.

In this chapter we are going to explore the declination of the ECS pattern in the Mass System via the core plugin MassEntity and the additional features available through the addition of standalone plugins offerd by Epic Games.

3.1 MassEntity

At the heart of Mass system lies MassEntity, the core module responsible for the framework's low-level heavy lifting. It manages the execution pipeline, entity lifecycle and the specialized memory layouts required for high-performance computing. Initially introduced as a plugin, MassEntity was integrated into the core engine as of UE5.5, with the standalone plugin version now deprecated.

Within MassEntity, we find specialized implementation of the three pillars of the ECS pattern:

- Entities: Unique identifiers that represent individual simulated objects
- Fragments: Logic-less data structures that serve the role of Components, storing the entity's data
- Processors: Highly optimized logic units that act as Systems, operating on specific sets of Fragments.

In the following sections, we will present an overview of the software architecture of Mass Entity, focusing on data structures, memoy organization and logic execution

to illustrate how these core ECS concepts are implemented within framework. Following this high-level overview, we will dive deeper in the analysis of each element, providing practical code examples to demonstrate their implementation and interaction.

3.1.1 Entities

In the Mass framework, an Entity is a lightweight, unique identifier rather than a data-carrying object. It serves as a hook to associate various fragments within the Mass database.

When an entity is created, the `MassEntitySubsystem` assigns it an index and a serial number used to prevent dangling references if the entity is destroyed. This abstraction allows the engine to treat entities as transient, high-performance entries in a table.

3.1.2 Memory Organization: Archetypes and Chunks

In Mass Framework, an Archetype is the logical grouping of entities sharing an identical composition of Fragments and Tags (`FMassArchetypeData`). As entities dynamically gain or lose fragments at runtime, they are migrated between Archetypes. This structural consistency allows the engine to bypass the pointer-chasing common in Object-Oriented Programming (OOP) in favor of linear memory access.

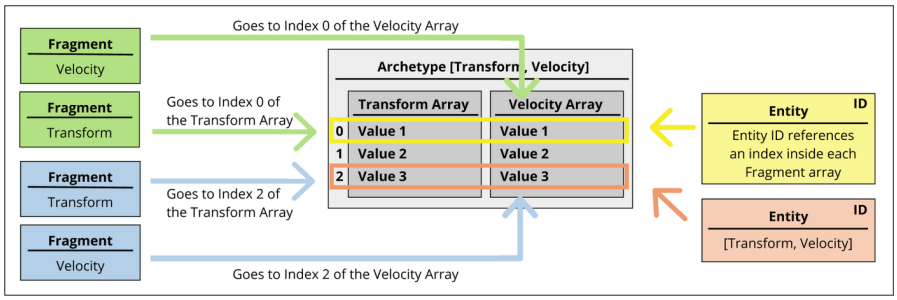

Figure 3.1: Logical grouping of entities into Archetypes based on fragment composition.

To optimize for CPU cache coherency, Mass organizes Archetype data into Chunks (`FMassArchetypeChunk`). A Chunk is a fixed-size memory block that serves as the fundamental unit of both memory allocation and parallel execution. Each Archetype holds an array of Chunks with fragment data and each chunk contains fragments referring to a subset of entities belonging to the archetype.

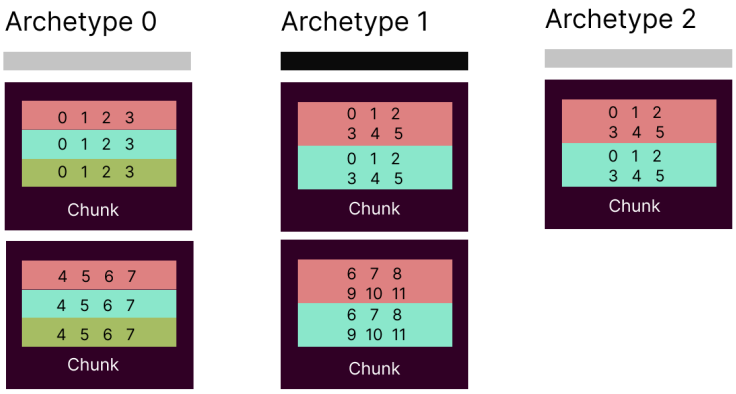

Figure 3.2: Logical organization of Archetype-related data in chunks. Different colors of rows correspond to different fragments while the numbers refer to the Entity ID

To facilitate rapid filtering and query matching, the archetype structure incorporates a bitset representation of its composition. Each bit in this mask corresponds to a specific fragment or tag type. If a bit is set, the archetype contains that component. This bitmasking approach allows the `MassEntitySubsystem` to perform constant-time validation when determines if an archetype satisfies the requirements of a processor's `FMassEntityQuery`.

The default size of the memory block is defined in UMassEntitySettings and it is equal to 128 KB. This size has been established based on current generation caches and could change in future versions of the engine.

Rather than storing all fragments of a single type for an entire Archetype in one massive array, Mass partitions the Archetypes into these 128 KB blocks. Within each chunk, data is organized as struct of arrays, assigning a block of memory to each type of Fragment and Tag instances.

By sizing chunks to fit within modern L2 or L3 caches, Mass ensures that when a processor requests multiple fragments for an entity, the entire row of data is likely already in the cache.

Furthermore, Chunks facilitate safe multi-threading. the MassEntitySubsystem can distribute individual chunks across different CPU cores via the Task Graph, ensuring that no two threads attempt to write to the same memory block simultaneously.

Because the ChunkMemorySize is a fixed constant, the entity density is inversely proportional to the total size of the fragments. Having slim entities means that we can fit more entities in the chunk, improving cache efficiency by reducing the management overhead. When designing fragments, developers should aim for

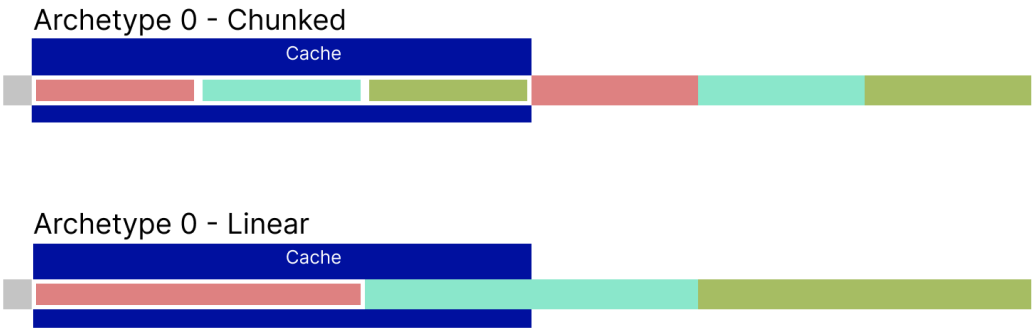

Figure 3.3: The image shows why it is convenient to have chunks sized to fit the cache rather than a single array structure for the whole Archetype.

granularity. Large monolithic fragments reduce the number of entities per chunk, potentially dropping the density to a point where the overhead of chunk-level iteration negates the performance gains of the ECS architecture.

3.1.3 Mass Execution Environment

To coordinate the complex relationship between archetypes, chunks, and entities, Mass utilizes a centralized management layer consisting of the `UMassEntitySubsystem` and the `FMassEntityManager`.

The `FMassEntityManager` acts as the low-level data authority. It maintains the global registry of all active archetypes and provides the logic for structural changes, such as adding or removing fragments and archetype migration. By centralizing these operations, the manager ensures that memory remains contiguous and that entity handles (`FMassEntityHandle`) remain valid throughout their lifecycle.

Surrounding the manager is the `UMassEntitySubsystem`, a `UWorldSubsystem`[1]. that provides a stable API for the rest of the engine. It facilitates the baking of `FMassEntityTemplates` and manages the lifecycle of the simulation relative to the `UWorld`. Together, these components serve as the backbone of the framework, translating high-level commands into the low-level memory operations that define the Mass Framework's performance characteristics.

[1]A World Subsystem in Unreal Engine is a singleton-like class that is automatically managed by the engine and tied directly to the lifecycle of a `UWorld` instance.

3.1.4 Entity Processing and Interaction

To maintain the integrity of the linear memory chunks during multi-threaded execution, Mass employs a deferred-mutation pattern via the Mass Command Buffer. Structural changes, such as spawning entities, destroying them, or altering their archetype through the addition or removal of fragments, are not executed immediately by processors. Instead, these requests are queued as Commands. At designated synchronization points in the frame, the `FMassEntityManager` flushes these buffers, performing the memory-intensive migrations in a batch. This prevents race conditions and ensures that pointers to fragment data remains valid throughout the duration of a processing task.

While processors typically operate on entire batches of entities, individual entity interaction is facilitated through the Entity View (`FMassEntityView`). Because fragments are stored in raw memory buffers, they cannot be accessed through standard pointer dereferencing from an entity handle. The Entity View acts as a transient window into a specific entity's data, resolving the `FMassEntityHandle` into a structured set of pointers for its constituent fragments. This provides a type-safe interface for logic that must operate outside of the standard batch-processing pipeline without sacrificing the performance benefits of the underlying ECS architecture.

3.1.5 Fragments and Tags

Fragments constitute the atomic data units of the Mass Framework. Unlike traditional class properties, fragments are implemented as lightweight `USTRUCTs` inheriting from `FMassFragment` and contain no logic, serving strictly as data containers. By decoupling data from behavior, Mass allows for highly flexible entity composition and optimal memory alignment.

While generic fragments are unique to each entity, the framework provides specialized fragment types to optimize memory footprint and group-level data management.

Entity Fragments

The standard fragment type stored in rows within a chunk. Each entity has its own instance of this data (e.g. `FTransformFragment`), allowing for unique per-entity state.

```

USTRUCT()
struct FSimpleMovementFragment : public FMassFragment
{
  GENERATED_BODY()
```

```
  FVector InitialTarget;
  float Velocity;
};
```

The design of individual fragments should strictly adhere to the principle of locality, where data is grouped according to its specific access pattern or functional aspect. For instance, a `FSimpleMovementFragment`, such as the one in the code example above, should ideally only contain variables relevant to spatial translation, such as `InitialTarget` and `Velocity`.

Integrating unrelated variables into a single monolithic fragment introduces several architectural inefficiencies:

1. **Cache Line Pollution:** CPUs fetch data from memory in fixed-size lines. If a movement processor iterates through a fragment that contains many variables to access velocity data, the CPU is forced to load the adjacent, unrelated variables into the cache. This results in polluted cache lines where a significant portion of the high-speed memory is occupied by data that will not be necessary for execution, effectively wasting memory bandwidth.

2. **Decreased Entity Density:** Because `ChunkMemorySize` is a fixed constant, the number of entities that can coexist within a single chunk is inversely proportional to the total size of its fragments. Increasing the memory footprint of a fragment directly reduces entity density, necessitating more frequent chunk-switching and increasing the management overhead per entity.

3. **Parallelization Constraints:** The Mass Framework achieves high performance through concurrent execution, where different processors can operate on different fragments of the same entity simultaneously. If disparate variables, such as `Health` and `Velocity`, are defined in a single fragment, a processor modifying health will effectively lock the entire fragment. This creates a data dependency that prevents a movement processor from accessing the velocity data in parallel, forcing sequential execution or requiring complex synchronization overhead that negates the benefits of the ECS architecture.

4. **Structural Change Overhead:** Entities in Mass are dynamic; adding or removing fragments triggers a migration to a new Archetype. This process involves a memory copy of the entity's existing fragments. Monolithic fragments significantly increase the volume of data that must be shuffled during these structural changes, even if the modification only involves a single, small variable.

Chunk Fragments

These fragments (`FMassChunkFragment`) are stored once per chunk rather than once per entity. They are ideal for data that is identical for every entity within a specific memory block, such as Level-of-Detail (LOD) information or spatial bounds. By updating a single chunk fragment, a processor can effectively update the state of hundreds of entities simultaneously.

Shared Fragments

Shared fragments (`FMassSharedFragment`) represent data that is common across multiple chunks or even multiple archetypes. They are stored in a central registry and referenced via a handle, making them highly efficient for constant data, such as a shared mesh reference or a global movement configuration, that rarely changes. Here is an example of shared fragment that contains the Player's position:

```

USTRUCT(BlueprintType)
struct FCurrentPlayerPositionFragment : public FMassSharedFragment
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    FVector PlayerPosition;
};
```

Tags

Implemented via `FMassTag`, tags are zero-sized fragments that carry no data, having no member properties. They exist solely as bits within the archetype's `FMassTagBitSet`. They are used for high-speed categorization and filtering, allowing processors to include or exclude entities based on boolean-like markers without increasing memory consumption.

To define a custom tag, a developer simply inherits from `FMassTag`, ensuring the struct remains empty:

```

USTRUCT()
struct FDeadTag : public FMassTag
{
    GENERATED_BODY()
};
```

3.1.6 Traits

While fragments and tags provide the low-level data structures for the ECS, traits serve as the high-level configuration layer that bridges the gap between the Unreal Editor and the ECS backend. A trait acts as a high-level recipe that specifies which fragments, tags and shared data should be attached to an entity. This abstraction allows designers to compose complex agents behavior, such as crowd movement, avoidance or visualization, without directly interacting with the underlying implementation.

A trait is implemented in C++ by inheriting from `UMassEntityTrait` and overriding the `BuildTemplate` function:

```

UCLASS(meta = (DisplayName = "NPC Horde Movement"))
class NPCHORDE_API UNPCMovementTrait : public UMassEntityTraitBase
{
    GENERATED_BODY()

protected:
    virtual void BuildTemplate(FMassEntityTemplateBuildContext& BuildContext, const UWorld& World) const override
    {
        FMassEntityManager& EntityManager =
                UE::Mass::Utils::GetEntityManagerChecked(World);

        // Ensuring another trait provides this fragment
        BuildContext.RequireFragment<FTransformFragment>();

        // Adding a Fragment
        BuildContext.AddFragment<FSimpleMovementFragment>();

        // Adding a Tag
        BuildContext.AddTag<FMovingTag>();

        // Creating and adding a shared fragment
        const FConstSharedStruct RandomMovementFragment = EntityManager.GetOrCreateConstSharedFragment(RandomMovement);
        BuildContext.AddConstSharedFragment(RandomMovementFragment);
    }

    UPROPERTY(Category = "Movement", EditAnywhere, meta = (EditInline))
    FMassMovementParameters RandomMovement;
};
```

When the `MassEntitySubsystem` processes a trait, it executes the `BuildTemplate` function. During this phase, the trait contributes with its components to the entity template adding necessary fragments and tags to the archetype definition.

Traits are primarily utilized during the construction of an `FMassEntityTemplate`, which is authored via a `UMassEntityConfigAsset`. This specialized Data Asset acts as a high-level container where designers can aggregate various traits to define an entity's composition and behavior. They often contain parameters that can be edited in the editor tab which are then used to populate the initial values of the fragments.

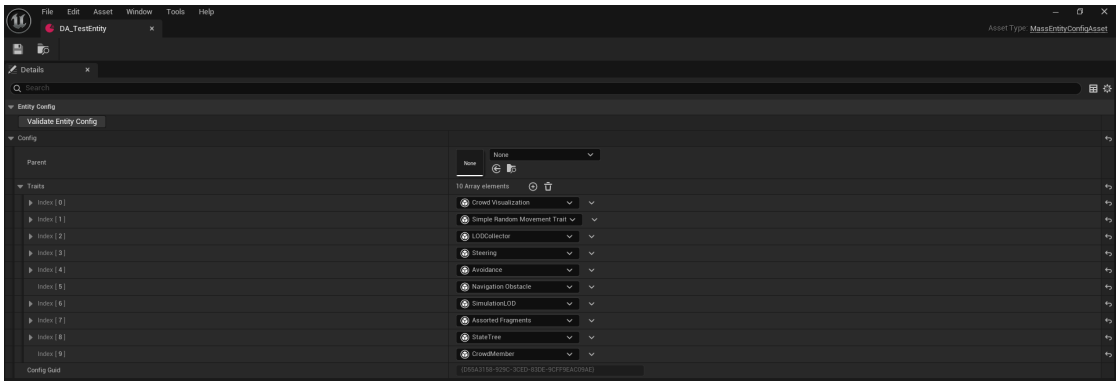

Figure 3.4: Example of Mass Entity Config Asset defining an Entity via editor. It is possible to create one by clicking on the "Add" button in the content drawer, selecting Data Asset and then choosing Mass Entity Config Asset.

While custom C++ traits offer the most control, the Mass Framework provides versatile built-in traits to streamline the development workflow. A notable example is the Assorted Fragments Trait (`UMassAssortedFragmentsTrait`). This trait utilizes an array of `FInstancedStructs`, allowing designers to add fragments directly within the Unreal Editor.

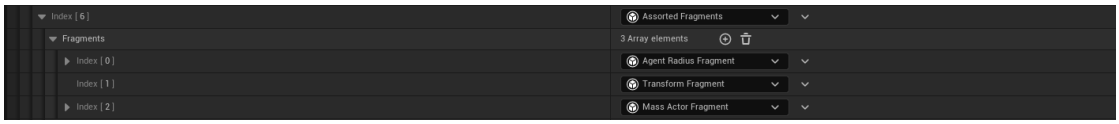

Figure 3.5: Example of Assorted Fragment trait use.

Unlike standard Actor Components, traits do not exist at runtime once the entity has been spawned but are Editor-time construction tools. Once the simulation begins, the trait's responsibility ends, and the entity exists purely as a collection of fragments and tags. This ensures that the high-level flexibility of the Unreal Editor does not compromise the efficiency of the simulation.

3.1.7 Processors and Queries

The System element of the ECS pattern is represented in Unreal Engine by Processors.

Processors are stateless classes (inheriting from `UMassProcessor`) that house the simulation logic. They are designed to be data-agnostic, meaning they do not operate on individual entities one by one, but rather on streams of data provided by the Entity Manager.

By default, any class deriving from `UMassProcessor` is automatically registered with the Mass Framework. To ensure synchronization with the rest of the Unreal Engine ecosystem (such as Physics, Animation, and Networking), the Mass Framework organizes processors into Processing Phases. Each phase is mapped to a specific `ETickingGroup`, which dictates when the Mass logic will execute relative to the rest of the engine's frame logic.

By default, processors are added to the `PrePhysics` phase, but developers can redistribute them to optimize parallelization or satisfy data dependencies.

Table 3.1: Unreal Engine 5 Mass Processing Phases and Tick Groups

Processing Phase	Unreal Tick Group	Description
PrePhysics	`TG_PrePhysics`	The default phase. Used for logic that sets up movement or state before the physics engine runs.
StartPhysics	`TG_StartPhysics`	A specialized phase that marks the beginning of the physics simulation.
DuringPhysics	`TG_DuringPhysics`	Logic that runs concurrently with the physics simulation, ideal for non-colliding visual logic or calculations that don't depend on current-frame physics results.
EndPhysics	`TG_EndPhysics`	A phase used to finalize physics-related calculations.
PostPhysics	`TG_PostPhysics`	Used for logic that depends on physics results, such as updating transforms based on collision or rigid body movement.
FrameEnd	`TG_LastDemotable`	A catch-all phase for cleanup or low-priority logic that can be deferred to the end of the frame.

The System element of the ECS pattern is represented in Unreal Engine by Processors.

Processors are stateless classes that house the simulation logic. They are designed to be data-agnostic, meaning they do not operate on individual entities one by one, but rather on streams of data.

They use EntityQueries to define their data requirements (e.g. "Provide all entities that have a Velocity Fragment but do not have and Enemy Tag"). The query fetches the data in batches from the Chunks, allowing the Processor to execute logic across thousands of entities with maximum efficiency and high potential for parallelization.

Accordingly to the memory organization articulated in the previous section, processors cover a chunk of entities in an archetype. The execution flow of Mass Processors follows this steps:

1. Each processor starts by configuring an entity query which adds requirements that will be used for filtering entities. Requirements can be put on tags, fragments, shared fragments, chunk fragments and even subsystems.

2. Processors then batch the updates for the chunk of entities by calling ForEachEntityChunk. This is where Chunk Fragments are use, since a single chunk fragment can be used across all the entities in the chunk.

3. The MassEntityQuery matches the requirements with the archetypes and can filter out chunks of matching archetypes based on chunk fragment filters. Requirements are usually put on the presence of a given tag or fragment in the archetype, but they can be also used to filter based on the absence of them.

4. After filtering, the MassEntityQuery triggers a function on each chunk of entities, and the individual entities of the chunk can be accessed via the FMassExecutionContext. Most of the plugin code makes use of lambda expressions when executing ForEachentityChunk.

3.2 Additional Mass Plugins

On top of the MassEntity core, Epic has developed several plugins that enable useful processors and traits for given known tasks. All these plugins in Unreal Engine 5.6 are marked as experimental, meaning that they could be subject to changes in the next future and should be used with care for projects ment for shipping.

These plugins are the result of the work done on the CitySample tech demo, where Mass was used to create massive crowd and vehicle simulations.

From here follows a brief exposition of the features provided by each of them in UE 5.6.

3.2.1 MassGameplay

Provides gameplay integration such as spawning entities in the world, replication support, level-of-detail (LOD) management, movement basics and visualization of entities spatially. It supports representing Mass entities as Actors if needed.

The plugin contains the following subsystems that build directly on top of MassEntity functionalities:

- Mass Representation Subsystem: responsible for managing the different visual aspects of the Mass Entities. It can recycle and pool spawned Actors automatically
- Mass Spawner Subsystem: spawns and manages the Entities based on the MassSpawner objects and procedural calls
- Mass LOD Subsystem: calculates the Level Of Detail (LOD) necessary for each Mass Entity. It outputs four LOD values: High, Medium, Low and Off
- Mass Replication Subsystem: replicates entities over the network in a client-server manner. It performs one-way replication from the server to its clients. This subsystem is considered experimental for Unreal Engine 5.1 and requires a C++ implementation to replicate custom values
- Mass StateTree Subsystem: used to integrate the StateTree system, a general-purpose hierarchical state machine, with Mass Entity. It allows us to configure StateTrees for each entity, and update each entity's StateTree based on signals sent from other Mass systems
- Mass Signals Subsystem: provides a way to send a named signal to tell an entity that it has some processing to do. This subsystem is currently used in Mass Statetree to wake it up when it has to perform some processing
- Mass Movement Subsystem: defines a simple movement model for Mass agents. The Fragments and Processors are set up so that other Traits can modify the velocity or force directly. For example, when steering the agent, the system may set the initial steering force and avoidance can alter it to avoid collisions during movement
- Mass SmartObject Subsystem: used to integrate the SmartObject system with MassEntity. SmartObjects are objects placed in a level that AI Agnets and Players can interact with: they are part of a global database and use a splatial partitioning structure. At a high level, Smart Objects represent a set of activities in the lecel that can be used through a reservation system. Mass SmartObject subsystem provides the requires Traits, Fragments and

Processors to perform Smart Object queries and execute simple behaviors on agents represented by Mass Entities.

3.2.2 MassAI

This plugin adds AI-specific logic such as behavior tree integration via StateTree, navigation via ZoneGraph, agent avoidance and AI replication in order to enable autonomous agents.

It is composed by the following modules:

- MassAIBehavior: implements core AI logic including behavior execution, State Trees integration, decision making processes, behavior fragments and processors managing AI state and actions
- MassAIDebug: provides debugging tools and integrations, such as gameplay debugger support, visualization of AI internal states, and diagnostics specific to MassAI entities to assist development and troubleshooting
- MassAIReplication: manages network replication of MassAI entities, handling entity state synchronization between server and clients with relevancy and LOD systems to optimize bandwidth usage for large AI populations
- MassAITestSuite: contains automated tests and validation tools ensuring the correctness and stability of MassAI functionalities during development and continuous integration

3.2.3 MassCrowd

Specializes in crowd simulation with custom visualization and navigation processors, built on top of previous plugins. This was used in projects like City Sample and The Matrix Awakens demo.

The module contains processors that handle crowd movement, avoidance and visualization tailored for large groups of Mass Entities.